Universidad Nacional de Tucumán
Facultad de Ciencias Exactas y Tecnología
Computación Científica

Estimación de las constantes π y e mediante el Método de Monte Carlo

Implementación serial en C y paralela con MPI
Evaluación Parcial 1 — Proyecto 1

Integrantes
Ason, Belén
Cano Arce, Valentina
Caballero, Leonardo

Repositorio: github.com/belenason/pi-y-e-metodo-montecarlo
Octubre de 2026

Resumen

En este trabajo se estiman las constantes π y e mediante el Método de Monte Carlo y se estudia una implementación paralela en C con MPI. La estimación de π se basa en la proporción de puntos aleatorios que caen dentro de un cuarto de círculo, y la de e, en la cantidad promedio de números uniformes que es necesario sumar para superar 1. Las N simulaciones se reparten de forma estática entre P procesos, que solo se comunican al final mediante una reducción. Se realizaron 1.700 ejecuciones en una computadora personal (4 núcleos físicos) y en un nodo de clúster (48 CPU asignadas), con N entre 10^5 y 10^9. El error disminuye como $1/\sqrt{N}$, con pendientes medidas de −0,505 (π) y −0,497 (e), y no depende de la cantidad de procesos. En el clúster, el *speedup* es prácticamente ideal hasta 48 procesos para $N \geq 10^8$ (eficiencia de 0,98 para π), mientras que en la computadora personal no supera 2,6 (π) y 4,4 (e), un límite impuesto por la plataforma y no por el algoritmo.

Palabras clave: Monte Carlo, computación de alto rendimiento, MPI, *speedup*, eficiencia paralela, convergencia.

Índice

1. Introducción **4**

2. Objetivos **5**

3. Fundamentos teóricos **6**

3.1. Método de Monte Carlo 6

3.2. Estimación de π 6

3.3. Estimación de e 7

3.4. Convergencia y error 9

3.5. Métricas de rendimiento paralelo 11

4. Implementación **12**

4.1. Organización del código 12

4.2. Versión serial 12

4.3. Estrategia de paralelización 14

4.4. Números aleatorios en paralelo 15

4.5. Comunicación entre procesos 15

4.6. Medición del tiempo 16

5. Diseño experimental **18**

5.1. Sistemas utilizados 18

5.2. Variables y parámetros 18

5.3. Procedimiento de medición 19

5.4. Tratamiento estadístico 19

6. Resultados y análisis **20**

6.1. Precisión: convergencia y error 20

6.2. Tiempos de ejecución 22
6.3. Speedup y eficiencia en el clúster 23
6.4. Speedup y eficiencia en la PC 25
6.5. Comparación entre sistemas 26
6.6. Limitaciones del estudio 27
7. Conclusiones 28
8. Mejoras futuras 29
8.1. Generador xoshiro256** con salto por proceso 29
8.2. Un número aleatorio por punto en π 29
8.3. Aritmética entera 29
8.4. Bucle sin saltos impredecibles en e 29
8.5. Error estándar en la salida 30
8.6. Mejoras en el procedimiento experimental 30
8.7. Otras alternativas consideradas 31
9. Bibliografía 32
Anexo A. Tabla completa de resultados 33
Anexo B. Gráficos complementarios 35

1. Introducción

Este proyecto consiste en la estimación de las constantes matemáticas π y e mediante el Método de Monte Carlo, aplicando técnicas de Computación de Alto Rendimiento (HPC, por sus siglas en inglés) en el lenguaje C con el estándar MPI, mediante la implementación Open MPI, para analizar y mejorar el rendimiento de los cálculos. El objetivo de este proyecto no se limita a obtener una aproximación de estas constantes, sino que también busca estudiar cómo se comporta el problema al aumentar el número de simulaciones y al distribuir el trabajo entre diferentes procesos.

El Método de Monte Carlo es una técnica numérica que utiliza la generación de muestras aleatorias y el análisis estadístico de sus resultados para obtener aproximaciones a problemas deterministas. Su principal característica es que, en lugar de buscar directamente una solución exacta mediante un procedimiento determinista, realiza una gran cantidad de simulaciones aleatorias y utiliza el comportamiento conjunto de estas muestras para aproximar el resultado buscado.

La elección de este método resulta especialmente adecuada para los objetivos del proyecto debido a que las simulaciones pueden realizarse de manera independiente entre sí. Esto permite distribuir una gran cantidad de simulaciones entre diferentes procesos y unidades de procesamiento, haciendo que el problema sea apropiado para estudiar técnicas de paralelización. Además, la cantidad de simulaciones puede modificarse para analizar cómo influye en la precisión de los resultados, la convergencia y el tiempo de ejecución.

Si bien existen otros métodos para calcular o aproximar π y e, como series matemáticas, métodos de integración numérica o algoritmos iterativos, la elección de Monte Carlo no se basa únicamente en su capacidad para aproximar las constantes, sino también en que permite estudiar de manera directa la distribución del trabajo y el comportamiento de una implementación paralela.

Para el desarrollo del proyecto se implementó primero una versión serial y luego una versión paralela del problema. Sobre ambas se realizó un estudio experimental variando el número de simulaciones N y la cantidad de procesos P en dos sistemas de cómputo distintos: una computadora personal y un nodo de un clúster. Cada configuración se ejecutó múltiples veces y se utilizó la mediana de los tiempos para reducir la influencia de variaciones ocasionales del sistema, lo que permite analizar la convergencia del error, el *speedup* y la eficiencia paralela.

Finalmente, se estudian y comparan los resultados obtenidos para determinar el comportamiento de la implementación, sus limitaciones y las posibles mejoras que podrían realizarse en futuros casos.

2. Objetivos

Objetivo general

Desarrollar y analizar una implementación paralela basada en el Método de Monte Carlo para estimar las constantes matemáticas π y e, estudiando la precisión de las aproximaciones y el comportamiento del programa en términos de rendimiento computacional.

Objetivos específicos

- Investigar el funcionamiento del Método de Monte Carlo y su aplicación para la estimación de π y e.
- Diseñar e implementar una estrategia de paralelización adecuada para el problema.
- Distribuir las simulaciones entre diferentes procesos de manera equilibrada.
- Analizar cómo influye la cantidad de simulaciones en la precisión de las estimaciones.
- Verificar experimentalmente la convergencia de las estimaciones y la tasa teórica del error, $1/\sqrt{N}$.
- Analizar el error obtenido para diferentes cantidades de simulaciones.
- Medir los tiempos de ejecución de las versiones serial y paralela.
- Calcular el *speedup* y la eficiencia obtenidos al utilizar diferentes cantidades de procesos.
- Analizar el comportamiento de la implementación al ejecutarla en diferentes sistemas de cómputo.
- Realizar múltiples ejecuciones de cada experimento y utilizar la mediana de los tiempos para el análisis del rendimiento.
- Identificar las limitaciones de la implementación y proponer posibles mejoras para futuros casos.

3. Fundamentos teóricos

3.1. Método de Monte Carlo

El Método de Monte Carlo es una técnica numérica que utiliza el muestreo aleatorio y métodos estadísticos para obtener aproximaciones a problemas deterministas. En lugar de calcular directamente una solución exacta mediante una fórmula cerrada, se plantea un experimento probabilístico cuyo valor esperado coincida con el resultado buscado, se realizan numerosas simulaciones aleatorias y se utiliza su promedio para aproximar la solución [1], [2].

Como el resultado surge del comportamiento estadístico de una gran cantidad de muestras, la cantidad de simulaciones realizadas (N) tiene una influencia directa sobre la precisión. La Ley de los Grandes Números garantiza que, al aumentar N, el promedio converge al valor esperado. Sin embargo, debido a la naturaleza aleatoria del método, distintas ejecuciones producen estimaciones ligeramente diferentes. La velocidad de esa convergencia se analiza en la sección 3.4.

Otra característica fundamental para este proyecto es la independencia entre las simulaciones: cada una puede realizarse sin conocer el resultado de las demás. Por lo tanto, el conjunto de N simulaciones puede dividirse en partes, asignarse a distintos procesos que las ejecutan simultáneamente y, al final, combinar los resultados parciales sin alterar el estimador, siempre que cada proceso utilice una secuencia de números pseudoaleatorios independiente de las demás. Por ejemplo, un millón de simulaciones pueden repartirse entre cuatro procesos que realizan 250.000 cada uno.

Esta estructura convierte a Monte Carlo en un problema trivialmente paralelo (*embarrassingly parallel*): los procesos no necesitan comunicarse durante el cálculo, solo al final. Por eso resulta especialmente adecuado para estudiar técnicas de Computación de Alto Rendimiento, ya que permite analizar con claridad la distribución del trabajo, el balance entre procesos y el *speedup* obtenido al utilizar diferentes cantidades de recursos.

3.2. Estimación de π

Para comprender cómo se puede estimar π mediante el Método de Monte Carlo, primero es necesario establecer una relación geométrica que permita obtener esta constante a partir de una proporción de áreas.

Si se inscribe un círculo de radio 1 en un cuadrado de lado 2, la proporción entre sus áreas es π/4. Por simplicidad, se trabaja únicamente con el primer cuadrante: un cuarto de círculo de radio 1, centrado en el origen, dentro del cuadrado de vértices (0, 0), (1, 0), (0, 1) y (1, 1). Debido a la simetría de ambas figuras respecto de los ejes cartesianos, la proporción entre las áreas se mantiene intacta. Además, las coordenadas quedan directamente en el intervalo [0, 1) que entrega el generador de números aleatorios, por lo que no es necesario transformarlas.

El cuadrado funciona como una región de referencia de área conocida, mientras que el cuarto de círculo introduce π a través de su área:

$$A_{cuadrado} = 1 \cdot 1 = 1, \quad A_{cuarto} = \frac{\pi \cdot 1^2}{4} = \frac{\pi}{4} \quad \Rightarrow \quad \frac{A_{cuarto}}{A_{cuadrado}} = \frac{\pi}{4} \tag{1}$$

¿Por qué tirar puntos en un cuadrado permite calcular π?

La idea fundamental del método es que, si los puntos se distribuyen uniformemente dentro del cuadrado, cada punto tiene la misma probabilidad de caer en cualquier zona. Por lo tanto, una región que ocupa una mayor superficie recibe, en proporción, más puntos, y la fracción de puntos que cae dentro de una región se aproxima a la fracción del área total que esa región ocupa. Aplicado al cuarto de círculo, si de N puntos generados M quedan dentro:

$$\frac{M}{N} \approx \frac{A_{cuarto}}{A_{cuadrado}} = \frac{\pi}{4} \quad \Rightarrow \quad \pi \approx 4 \cdot \frac{M}{N} \tag{2}$$

Es decir, no es el cuadrado por sí mismo el que permite calcular π: los puntos aleatorios permiten estimar la proporción entre ambas áreas sin tener que calcular directamente el área del cuarto de círculo.

Generación y clasificación de los puntos

En cada simulación se generan dos números aleatorios x e y, uniformes en [0, 1), que definen un punto ubicado dentro del cuadrado. El punto pertenece al cuarto de círculo si su distancia al origen, obtenida mediante el teorema de Pitágoras, no supera el radio: $\sqrt{(x^2 + y^2)} \leq 1$. Como ambos miembros son positivos, la condición equivale a:

$$x^2 + y^2 \leq 1 \tag{3}$$

Esta forma evita calcular una raíz cuadrada en cada simulación.

Por ejemplo, si se generan 1.000.000 de puntos y 785.400 de ellos quedan dentro del cuarto de círculo, la proporción es 0,7854 y la estimación resulta $\pi \approx 4 \cdot 0{,}7854 = 3{,}1416$. La estimación no coincide exactamente con el valor real de π porque se trabaja con una cantidad finita de puntos generados aleatoriamente, pero al aumentar la cantidad de simulaciones la proporción observada tiende a acercarse cada vez más a la proporción real entre las áreas (Figura 1, izquierda).

3.3. Estimación de e

La idea del método consiste en generar números aleatorios uniformemente distribuidos entre 0 y 1 y acumularlos hasta que la suma sea mayor que 1. La cantidad de números necesarios para superar 1 constituye el resultado de una simulación. A partir de la repetición de este procedimiento una gran cantidad de veces, se calcula el promedio de las cantidades obtenidas, que tiende al valor de e.

Procedimiento de una simulación

Se comienza con una suma inicial $S = 0$ y se le suman números aleatorios x_1, x_2, … uniformes en [0, 1) hasta que $S = x_1 + x_2 + \ldots + x_K > 1$. La cantidad K de números que fueron necesarios es el resultado de la simulación. Por ejemplo, si se generan 0,25; 0,18; 0,31 y 0,42, las sumas acumuladas son 0,25 → 0,43 → 0,74 → 1,16 > 1, por lo que $K = 4$. Otra simulación podría necesitar solamente 2 números, y otra 3, 5 o una cantidad diferente: el resultado de cada simulación es una variable aleatoria.

¿Por qué este procedimiento permite estimar e?

La relación con e surge de una propiedad matemática de este procedimiento: el valor esperado de K es exactamente e. Esto significa que, aunque una simulación individual dé como resultado 2, 3, 4 o cualquier otro entero, si se repite el experimento muchas veces el promedio de los resultados tiende a e. Si K_i es la cantidad de números necesarios en la simulación i, la estimación se obtiene mediante:

$$e \approx \hat{e}_N = \frac{1}{N} \sum_{i=1}^{N} K_i \tag{4}$$

La fórmula no significa que cada simulación produzca una aproximación de e: cada una produce solamente un número entero, y es el promedio de una gran cantidad de simulaciones el que permite aproximar la constante.

¿De dónde surge matemáticamente la relación con e?

La relación puede entenderse considerando la probabilidad de que sean necesarios más de k números para superar 1. Eso ocurre cuando la suma de los primeros k números no supera 1. Geométricamente, esa condición define un símplex de dimensión k dentro del hipercubo unitario, cuyo volumen es $1/k!$ [3]:

$$P(K > k) = P(x_1 + x_2 + \ldots + x_k \leq 1) = \frac{1}{k!} \tag{5}$$

Para una variable aleatoria que toma valores enteros no negativos, $E[K] = \sum_{k\geq 0} P(K > k)$. Por lo tanto:

$$E[K] = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \ldots = \sum_{k\geq 0} \frac{1}{k!} = e \tag{6}$$

En la implementación no es necesario calcular esta serie ni las probabilidades anteriores: el programa solamente realiza las simulaciones, cuenta cuántos números fueron necesarios en cada una y calcula el promedio.

A diferencia de la estimación de π, donde cada simulación consume siempre dos números aleatorios, aquí cada simulación consume una cantidad aleatoria de números (en promedio, e ≈ 2,72). Por lo tanto, el costo de cada simulación es variable, un aspecto relevante para el reparto del trabajo entre procesos (sección 4.3). La Figura 1 ilustra ambos experimentos.

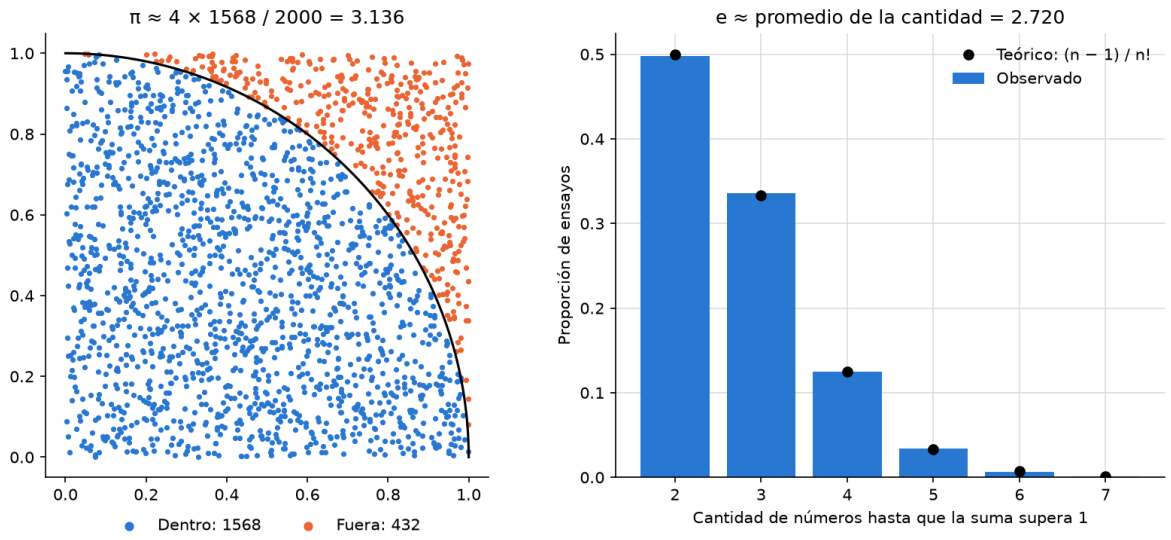


Figura 1. Ilustración de los dos experimentos con pocas muestras. Izquierda: 2.000 puntos aleatorios en el cuadrado unitario; los que caen dentro del cuarto de círculo (azul) permiten estimar π. Derecha: distribución de la cantidad de números uniformes necesarios para que su suma supere 1, frente a la probabilidad teórica $(n - 1)/n!$; su promedio estima e.

3.4. Convergencia y error

3.4.1. Convergencia

Ambas estimaciones son promedios de variables aleatorias independientes e idénticamente distribuidas. En el caso de π, cada punto es un ensayo con dos resultados posibles (dentro o fuera) y probabilidad de acierto $p = \pi/4$; la cantidad de aciertos M sigue una distribución Binomial(N, p) y el estimador es $\hat{\pi}_N = 4 \cdot M/N$. En el caso de e, el estimador es el promedio $\hat{e}_N$ de la Ec. 4. Por la Ley Fuerte de los Grandes Números, $\hat{\pi}_N \to \pi$ y $\hat{e}_N \to e$ cuando $N \to \infty$ [2].

3.4.2. Velocidad de convergencia

La Ley de los Grandes Números indica hacia dónde converge el método, pero no qué tan rápido lo hace. Si cada simulación tiene varianza σ^2, el promedio de N simulaciones independientes tiene varianza σ^2/N, de modo que su desviación estándar, que representa el error típico de una estimación, es:

$$\sigma_{\text{estimador}} = \frac{\sigma}{\sqrt{N}} \tag{7}$$

Para los dos estimadores de este proyecto, la varianza por simulación puede calcularse exactamente:

$$\sigma_\pi^2 = 16 \cdot p(1 - p) = \pi(4 - \pi) \approx 2{,}697 \quad (\sigma_\pi \approx 1{,}642) \tag{8}$$

$$\sigma_e^2 = \mathrm{Var}(K) = e(3 - e) \approx 0{,}766 \quad (\sigma_e \approx 0{,}875) \tag{9}$$

Por lo tanto, ambos métodos convergen con una tasa $O(1/\sqrt{N})$. Esta convergencia es lenta: para reducir el error a la mitad es necesario cuadruplicar N, y para ganar un dígito decimal de precisión, es decir, dividir el error por 10, es necesario multiplicar N por 100. La Tabla 1 muestra el error típico esperado para los valores de N utilizados en los experimentos.

Tabla 1. Error típico teórico $\sigma/\sqrt{N}$ de cada estimación para los valores de N utilizados.

N	$\sigma_\pi/\sqrt{N}$	$\sigma_e/\sqrt{N}$
10^5	$5{,}19 \cdot 10^{-3}$	$2{,}77 \cdot 10^{-3}$
10^6	$1{,}64 \cdot 10^{-3}$	$8{,}75 \cdot 10^{-4}$
10^7	$5{,}19 \cdot 10^{-4}$	$2{,}77 \cdot 10^{-4}$
10^8	$1{,}64 \cdot 10^{-4}$	$8{,}75 \cdot 10^{-5}$
10^9	$5{,}19 \cdot 10^{-5}$	$2{,}77 \cdot 10^{-5}$

Además, por el Teorema Central del Límite, el error de una estimación se distribuye aproximadamente como una normal, por lo que se espera que cerca del 95 % de las ejecuciones presente un error menor que $2\sigma/\sqrt{N}$; esta propiedad se verifica experimentalmente en la sección 6.1. Para un mismo N, la estimación de e es casi el doble de precisa que la de π ($\sigma_e/\sigma_\pi \approx 0{,}53$), aunque cada una de sus simulaciones consume más números aleatorios.

Esta lentitud de la convergencia es la que motiva la paralelización: cada dígito adicional multiplica por 100 el trabajo necesario. Existe, entonces, un compromiso entre precisión y costo computacional: aumentar N produce estimaciones estadísticamente más estables, pero también aumenta la cantidad de trabajo que debe realizar el sistema.

3.4.3. Fluctuaciones de los resultados

La disminución del error no es estrictamente monótona en cada ejecución: una ejecución con $N = 10^4$ puede, por azar, quedar más cerca del valor real que otra con $N = 10^5$. Esto no contradice la convergencia del método, que describe una tendencia estadística y no una garantía para cada ejecución individual. Por ese motivo, en este trabajo cada configuración se ejecuta varias veces con semillas distintas y el error se resume con medidas estadísticas sobre todas las ejecuciones.

3.4.4. Métricas de error

Para cada ejecución se calcula el error absoluto respecto del valor de referencia, utilizando como referencia π y e con 20 dígitos significativos, definidos como constantes en el código:

$$E_{abs} = |\ \text{valor estimado} - \text{valor real}\ | \tag{10}$$

Para resumir las R ejecuciones realizadas con un mismo N se utiliza la raíz del error cuadrático medio (RMS):

$$E_{RMS}(N) = \sqrt{\left(\frac{1}{R}\sum_{i=1}^{R} E_{abs,i}^{\,2}\right)} \tag{11}$$

Esta medida se elige porque estima directamente la desviación estándar del estimador y, por lo tanto, puede compararse con el valor teórico $\sigma/\sqrt{N}$ de la Tabla 1. Por último, para comparar ejecuciones con distintos N se usa el error normalizado $E_{abs}/(\sigma/\sqrt{N})$, cuyo valor RMS debería ser cercano a 1 si el método se comporta según la teoría.

3.5. Métricas de rendimiento paralelo

El rendimiento de la versión paralela se evalúa mediante el *speedup* y la eficiencia [4], [5]:

$$S_P = \frac{T_s}{T_P} \qquad E_P = \frac{S_P}{P} \tag{12}$$

donde T_s es el tiempo de la versión serial y T_P el de la versión paralela con P procesos. Como referencia T_s se utiliza un programa serial independiente, sin MPI, y no la versión paralela ejecutada con un solo proceso; así, el *speedup* no se ve favorecido si la versión MPI tuviera un costo adicional. El caso ideal es $S_P = P$ ($E_P = 1$). En la práctica, la comunicación, la sincronización y las limitaciones del hardware hacen que $S_P < P$. Valores con $S_P > P$ (superlineales) pueden aparecer por efectos de memoria caché o por errores de medición, y deben analizarse con cuidado.

La Ley de Amdahl [4] acota el *speedup* alcanzable cuando una fracción del programa no puede paralelizarse. Si f es la fracción paralelizable del tiempo serial:

$$S_P \leq \frac{1}{(1 - f) + f / P} \tag{13}$$

En este problema, la parte no paralelizable dentro del tiempo medido se reduce a combinar los resultados parciales y calcular la estimación final, operaciones cuyo costo no depende de N. Por lo tanto, f se acerca a 1 a medida que N crece, y se espera un *speedup* cercano al ideal para N grandes. En cambio, el costo de la comunicación crece con P, un efecto que la Ley de Amdahl no contempla y que se vuelve visible cuando el trabajo por proceso es pequeño. Los experimentos de este trabajo miden escalabilidad fuerte: N fijo y P variable.

4. Implementación

4.1. Organización del código

El código fuente se encuentra en el repositorio `github.com/belenason/pi-y-e-metodo-montecarlo` (Tabla 2). Para que la comparación entre versiones sea justa, el núcleo de cada estimación, es decir, el bucle de simulaciones, se escribió una sola vez, en un archivo de cabecera que incluyen tanto la versión serial como la paralela. De este modo, ambas versiones ejecutan exactamente el mismo código de cálculo, y la única diferencia entre ellas es cómo se reparte el trabajo.

Tabla 2. Organización del código fuente.

Archivo	Contenido
`montecarlo.h`	Lectura de argumentos, constantes de referencia y salida en formato CSV
`montecarlo_pi.h`, `montecarlo_e.h`	Núcleo de cálculo de cada estimación
`montecarlo_pi_serial.c`, `montecarlo_e_serial.c`	Versiones seriales
`montecarlo_pi_mpi.c`, `montecarlo_e_mpi.c`	Versiones paralelas (MPI)
`Makefile`	Compilación
`benchmark.sh`	Ejecución automatizada de los experimentos
`graficos.py`	Cálculo de métricas y generación de gráficos

4.2. Versión serial

Como primera etapa se desarrolló una versión serial de cada estimación. Esta versión permite verificar el funcionamiento de los métodos y obtener el tiempo de referencia T_s para calcular el *speedup*.

Parámetros. Cada programa recibe por línea de comandos la cantidad de simulaciones N y, opcionalmente, una semilla (por defecto, 12345). N y los contadores se almacenan en variables de tipo `long long` (64 bits), ya que los experimentos llegan a $N = 10^9$ y, en la estimación de e, el contador de números utilizados alcanza aproximadamente $e \cdot 10^9 \approx 2{,}7 \cdot 10^9$, un valor que supera el máximo de un entero de 32 bits ($\approx 2{,}1 \cdot 10^9$).

Generación de números aleatorios. Se utiliza la función `drand48()` de la biblioteca estándar de C, que devuelve valores de tipo `double` uniformes en [0, 1) a partir de un generador congruencial lineal de 48 bits [6]. Se prefirió frente a `rand()` porque el rango y la calidad de esta última dependen de la implementación: el estándar solo garantiza que `RAND_MAX` sea al menos 32.767. El generador se inicializa con `srand48(semilla)`. Utilizar una semilla explícita en lugar de la hora del sistema hace que cada experimento sea reproducible: con la misma semilla se obtiene exactamente la misma estimación.

Núcleo de cálculo. El Listado 1 muestra las dos funciones de cálculo. En π, por cada simulación se generan dos coordenadas y se incrementa un contador si se cumple la condición de la Ec. 3; la estimación es 4 · aciertos/*N*. En e, cada simulación suma números aleatorios hasta superar 1 y acumula en un contador la cantidad de números utilizados; la estimación es el total de números dividido por *N*.

Listado 1. Núcleo de cálculo de ambas estimaciones (`montecarlo_pi.h` y `montecarlo_e.h`).

```
static long long contar_aciertos_pi(long long n)
{
    long long aciertos = 0;

    for (long long i = 0; i < n; i++) {
        double x = drand48();
        double y = drand48();
        if (x * x + y * y <= 1.0) { // punto dentro del circulo
            aciertos++;
        }
    }
    return aciertos;
}

static long long contar_sumandos_e(long long n)
{
    long long sumandos = 0;

    for (long long i = 0; i < n; i++) {
        double suma = 0.0;
        while (suma <= 1.0) {
            suma += drand48();
            sumandos++;
        }
    }
    return sumandos;
}
```

Medición del tiempo. Se mide únicamente el núcleo de cálculo con `clock_gettime(CLOCK_MONOTONIC)`, un reloj de pared con resolución de nanosegundos que, a diferencia de `gettimeofday()`, no se ve afectado por ajustes de la hora del sistema. La lectura de argumentos, la inicialización del generador y la impresión de los resultados quedan fuera de la medición.

Salida. Cada ejecución imprime una línea en formato CSV con la constante estimada, *N*, *P*, la semilla, la estimación, el error absoluto y el tiempo. Este formato permite procesar automáticamente miles de ejecuciones.

Compilación. Los cuatro programas se compilan con `make`, que utiliza `gcc` para las versiones seriales y `mpicc` para las paralelas, con las mismas opciones de compilación (`-O2 -Wall -Wextra`), de modo que la diferencia de rendimiento no se deba al compilador. Por ejemplo, la versión serial de π se ejecuta como `./montecarlo_pi_serial 100000000 12345`.

4.3. Estrategia de paralelización

Análisis del código serial. En ambos programas, prácticamente todo el tiempo de ejecución se concentra en el bucle de simulaciones, cuyo costo crece linealmente con N. El resto del programa (lectura de argumentos, cálculo de la estimación y del error, impresión) tiene un costo constante y despreciable frente al bucle. Por lo tanto, la paralelización se concentra en ese bucle.

Descomposición del trabajo. Como las simulaciones son independientes (sección 3.1), se aplica una descomposición de dominio sobre el conjunto de simulaciones. Todos los procesos ejecutan el mismo programa, pero cada uno sobre una parte distinta de las simulaciones, siguiendo el modelo SPMD (*Single Program, Multiple Data*). Cada proceso calcula un resultado parcial (aciertos en π, números utilizados en e) y, al final, esos resultados se suman (Figura 2).

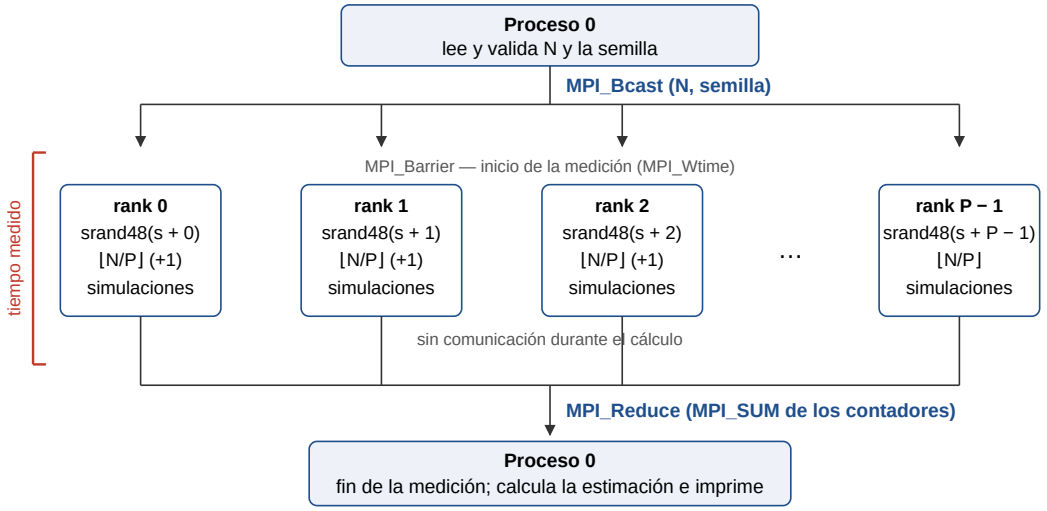


Figura 2. Esquema de la versión paralela: distribución de los parámetros, cálculo local independiente en cada proceso y reducción de los contadores en el proceso 0. La llave roja indica el intervalo de tiempo medido.

Distribución del trabajo. Las N simulaciones se reparten en bloques: cada proceso realiza $\lfloor N/P \rfloor$ simulaciones, y los primeros N mod P procesos realizan una más. Así se reparten todas las simulaciones aunque N no sea múltiplo de P, y la diferencia entre procesos es, como máximo, de una simulación.

Se eligió un reparto estático, decidido antes de comenzar el cálculo, en lugar de uno dinámico. En π, todas las simulaciones tienen el mismo costo, por lo que el reparto en partes iguales equilibra la carga exactamente. En e, el costo de cada simulación es variable (sección 3.3); sin embargo, cada proceso realiza millones de simulaciones, y la cantidad total de números que consume se concentra alrededor de $e \cdot n_{\text{local}}$, con una variación relativa del orden de $0{,}32/\sqrt{n_{\text{local}}}$. Con $n_{\text{local}} = 2 \cdot 10^7$ ($N = 10^9$ y $P = 48$), esa variación es de 0,007 %. Un reparto dinámico solo agregaría comunicación, sin ninguna ganancia.

Independencia entre la precisión y la cantidad de procesos. Distribuir las simulaciones no cambia la cantidad total de muestras: la estimación con P procesos utiliza las mismas N simulaciones que la versión serial, solo que ejecutadas en paralelo. Por lo tanto, la precisión depende de N y no de P: aumentar N tiene como objetivo mejorar la precisión, mientras que aumentar P tiene como objetivo

reducir el tiempo de ejecución. Esta separación, que se verifica experimentalmente en la sección 6.1, permite estudiar por un lado cuánto trabajo es necesario para obtener una determinada precisión y, por otro, cuánto puede reducirse el tiempo necesario para realizar ese trabajo.

4.4. Números aleatorios en paralelo

Para que las simulaciones de distintos procesos sean independientes, cada proceso debe utilizar una secuencia distinta de números pseudoaleatorios. Si todos usaran la misma semilla, generarían exactamente los mismos puntos, y la estimación con P procesos equivaldría a repetir P veces las mismas N/P simulaciones. En la implementación, cada proceso inicializa su generador con `srand48(semilla + rank)`, donde `rank` es el identificador del proceso.

Este esquema es simple y produce secuencias distintas en cada proceso, pero no garantiza formalmente que sean independientes. Todos los procesos recorren el mismo ciclo del generador, de período 2^{48}, a partir de puntos de partida que no se eligen para evitar solapamientos. Además, por la forma en que `srand48` construye el estado inicial, los 16 bits menos significativos del estado son iguales en todos los procesos [7]. Con los tamaños utilizados (como máximo, unos $2{,}7 \cdot 10^9$ números por proceso, muy por debajo del período de $\approx 2{,}8 \cdot 10^{14}$) no se espera que esto afecte los resultados, y la sección 6.1 muestra que el error no aumenta con P. Aun así, se trata de una limitación conocida, cuya solución se propone en la sección 8.

4.5. Comunicación entre procesos

La comunicación se limita al inicio y al final del cálculo (Listado 2):

- **Distribución de los parámetros.** Solo el proceso 0 lee y valida los argumentos. Si son inválidos, llama a `MPI_Abort`, de modo que todos los procesos terminen de forma ordenada. Si son válidos, envía N y la semilla al resto de los procesos con una única llamada a `MPI_Bcast`, empaquetando ambos valores en un arreglo de dos elementos.
- **Cálculo.** Durante las simulaciones no hay ninguna comunicación.
- **Combinación de resultados.** Cada proceso aporta su contador parcial a una operación `MPI_Reduce` con `MPI_SUM`, cuyo resultado queda en el proceso 0, que calcula la estimación final y la imprime. Se reducen los contadores enteros (aciertos o números utilizados) y no las estimaciones parciales de cada proceso: así, la suma es exacta, sin error de redondeo, y no es necesario ponderar los procesos que realizaron una simulación más que otros.

En total, cada ejecución intercambia un `MPI_Bcast` de 16 bytes y un `MPI_Reduce` de 8 bytes por proceso, independientemente de N. Las implementaciones de MPI realizan la reducción en forma de árbol, por lo que su costo crece aproximadamente con $\log_2 P$ [5]. Para N grandes este costo es despreciable frente al cálculo; para N pequeños, en cambio, puede volverse comparable, como se observa en la sección 6.3.

Listado 2. Estructura de la versión paralela de π (`montecarlo_pi_mpi.c`). La versión de e es idéntica, salvo por la función de cálculo.

```
int main(int argc, char *argv[])
{
    int rank, procesos;
    long long datos[2] = {0, 0};   // N y semilla

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &procesos);

    if (rank == 0 && !leer_argumentos(argc, argv, datos)) {
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    MPI_Bcast(datos, 2, MPI_LONG_LONG, 0, MPI_COMM_WORLD);

    long long n = datos[0];
    long long semilla = datos[1];
    long long n_local = n / procesos;
    if (rank < n % procesos) {
        n_local++;
    }

    srand48(semilla + rank);

    MPI_Barrier(MPI_COMM_WORLD);
    double inicio = MPI_Wtime();

    long long aciertos_local = contar_aciertos_pi(n_local);
    long long aciertos_total = 0;
    MPI_Reduce(&aciertos_local, &aciertos_total, 1, MPI_LONG_LONG,
               MPI_SUM, 0, MPI_COMM_WORLD);

    double fin = MPI_Wtime();

    if (rank == 0) {
        double estimacion = calcular_estimacion_pi(aciertos_total, n);
        imprimir_resultado(CONSTANTE_PI, n, procesos, semilla,
                           estimacion, fin - inicio);
    }

    MPI_Finalize();
    return 0;
}
```

4.6. Medición del tiempo

En la versión paralela, todos los procesos se sincronizan con `MPI_Barrier` antes de comenzar, de modo que inician el cálculo al mismo tiempo. El tiempo se mide con `MPI_Wtime()` desde ese punto hasta después de `MPI_Reduce`, y se informa el medido por el proceso 0. Como este proceso recibe el resultado de la reducción solo cuando todos los procesos aportaron el suyo, el tiempo medido incluye al proceso más lento.

Al igual que en la versión serial, la medición no incluye el lanzamiento de los procesos por `mpirun`, la inicialización de MPI, la distribución de los parámetros ni la finalización. De esta forma, ambas versiones miden lo mismo (el cálculo y, en la versión paralela, la combinación de los resultados), y

el *speedup* refleja el efecto de la paralelización. La contracara es que el tiempo total de una ejecución completa es mayor que el medido, y para *N* pequeños el lanzamiento de los procesos dominaría ese tiempo total (sección 6.6).

5. Diseño experimental

5.1. Sistemas utilizados

Los experimentos se realizaron en dos sistemas con características muy distintas (Tabla 3). La información de hardware se registró automáticamente en cada corrida, mediante `lscpu`, `nproc` y el límite de CPU del contenedor.

Tabla 3. Características de los sistemas utilizados.

	Computadora personal (PC)	Nodo de clúster
Procesador	Intel Core i5-10210U (1,6 GHz base; hasta 4,2 GHz con Turbo Boost; 15 W)	2 × AMD EPYC 7B12 (64 núcleos cada uno; hasta 3,3 GHz)
Núcleos físicos / lógicos	4 / 8 (Hyper-Threading)	128 / 128 (sin SMT); 48 CPU asignadas al contenedor
Caché L3	6 MiB	512 MiB en total
Nodos NUMA	1	2
Entorno	Linux sobre WSL 2 (máquina virtual en Windows)	Contenedor Linux con cuota de CPU (`cpu.max` = 4800000/100000, es decir, 48 CPU)
Compilación	gcc / mpicc (Open MPI) con `-O2 -Wall -Wextra`	
Opciones de `mpirun`	`--oversubscribe`	`--bind-to none --oversubscribe`
Valores de *P*	1, 2, 3, 4, 6, 8	1, 2, 4, 8, 12, 16, 24, 32, 48

En la PC se probaron hasta 8 procesos para observar el efecto de superar los 4 núcleos físicos. En el clúster, el contenedor dispone de 48 CPU, por lo que ese es el máximo de procesos con un núcleo por proceso; se incluyeron valores intermedios que no son potencias de 2 (12, 24 y 48) para observar el comportamiento cerca de ese límite. En el clúster se utilizó la opción `--bind-to none` porque, dentro de un contenedor, Open MPI no siempre puede fijar cada proceso a un núcleo; con esa opción, es el sistema operativo quien decide dónde ubicarlos [11].

5.2. Variables y parámetros

- **Constante estimada:** π y e (programas separados).
- **Cantidad de simulaciones:** $N \in \{10^5, 10^6, 10^7, 10^8, 10^9\}$, que cubre cinco órdenes de magnitud. El valor menor permite observar el efecto de los costos fijos de la paralelización, y el mayor produce tiempos seriales de unos 12 s (π) y 34 s (e), suficientemente largos para medir con estabilidad.
- **Cantidad de procesos:** según el sistema (Tabla 3).
- **Repeticiones:** 10 por cada combinación de constante, N y P, además de 10 ejecuciones de la versión serial por cada combinación de constante y N.
- **Semillas:** cada ejecución utiliza una semilla distinta: $100.000 \cdot r$ en la versión serial y $100.000 \cdot r + 1.000 \cdot P$ en la paralela, donde r es el número de repetición; a esta semilla cada proceso le

suma su `rank`. Los saltos entre semillas evitan que dos ejecuciones de una misma corrida compartan semilla.

En total se realizaron 700 ejecuciones en la PC y 1.000 en el clúster. En las tablas y el texto se utiliza coma decimal; los ejes de los gráficos, generados con matplotlib, utilizan punto decimal.

5.3. Procedimiento de medición

Las ejecuciones se automatizaron con el script `benchmark.sh`, que compila los programas, registra la información del sistema y recorre todas las combinaciones de parámetros, guardando una fila por ejecución en un archivo CSV. Las configuraciones se ejecutan intercaladas: en cada repetición se ejecuta la versión serial y, a continuación, la versión paralela con todos los valores de P. De este modo, si las condiciones del sistema cambian durante la corrida (temperatura, otras tareas), el cambio afecta de forma similar a todas las configuraciones, en lugar de sesgar a una sola.

Durante las mediciones no se ejecutaron otras aplicaciones y la PC permaneció conectada a la corriente. Luego, el script `graficos.py` procesa los archivos CSV, calcula las métricas y genera los gráficos. Todo el procedimiento puede reproducirse a partir del repositorio.

5.4. Tratamiento estadístico

Tiempos. Para cada configuración se toma la mediana de las 10 ejecuciones. La mediana es robusta frente a valores atípicos: una ejecución demorada por un proceso ajeno no la modifica, mientras que sí alteraría el promedio. El *speedup* se calcula con las medianas: S_P = mediana(T_s) / mediana(T_P).

Error. Para analizar la precisión se utilizan todas las ejecuciones con un mismo N (sección 3.4.4). Las ejecuciones con igual semilla y cantidad de procesos generan exactamente los mismos números en ambos sistemas y producen la misma estimación, por lo que se cuentan una sola vez; quedan así 1.200 ejecuciones distintas.

6. Resultados y análisis

6.1. Precisión: convergencia y error

La Figura 3 muestra las estimaciones obtenidas en todas las ejecuciones en función de N. En ambas constantes, la dispersión alrededor del valor real se reduce a medida que N aumenta, como predice la Ley de los Grandes Números.

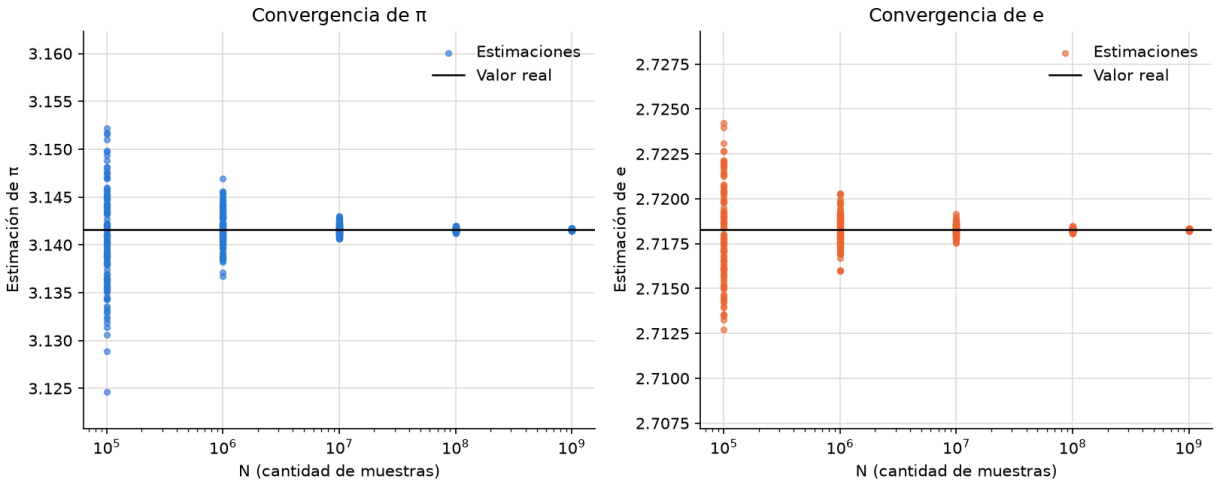


Figura 3. Estimaciones de π y e de todas las ejecuciones en función de N (escala logarítmica en N). La línea negra indica el valor real.

Para cuantificar la velocidad de convergencia, la Tabla 4 compara el error RMS medido con el valor teórico $\sigma/\sqrt{N}$, y la Figura 4 los representa en escala logarítmica, en la que una relación del tipo error $\propto N^{-1/2}$ aparece como una recta de pendiente −0,5. Las pendientes ajustadas son −0,505 para π y −0,497 para e, prácticamente iguales a la teórica. Además, los valores medidos difieren de los teóricos en menos de un 12 % en todos los casos, y en menos de un 4 % para $N \geq 10^8$.

Tabla 4. Error RMS medido y error típico teórico $\sigma/\sqrt{N}$ (120 ejecuciones distintas por constante y valor de N).

N	π medido	π teórico	e medido	e teórico
10^5	$5{,}25 \cdot 10^{-3}$	$5{,}19 \cdot 10^{-3}$	$2{,}57 \cdot 10^{-3}$	$2{,}77 \cdot 10^{-3}$
10^6	$1{,}82 \cdot 10^{-3}$	$1{,}64 \cdot 10^{-3}$	$8{,}73 \cdot 10^{-4}$	$8{,}75 \cdot 10^{-4}$
10^7	$5{,}55 \cdot 10^{-4}$	$5{,}19 \cdot 10^{-4}$	$3{,}04 \cdot 10^{-4}$	$2{,}77 \cdot 10^{-4}$
10^8	$1{,}62 \cdot 10^{-4}$	$1{,}64 \cdot 10^{-4}$	$8{,}52 \cdot 10^{-5}$	$8{,}75 \cdot 10^{-5}$
10^9	$5{,}29 \cdot 10^{-5}$	$5{,}19 \cdot 10^{-5}$	$2{,}68 \cdot 10^{-5}$	$2{,}77 \cdot 10^{-5}$

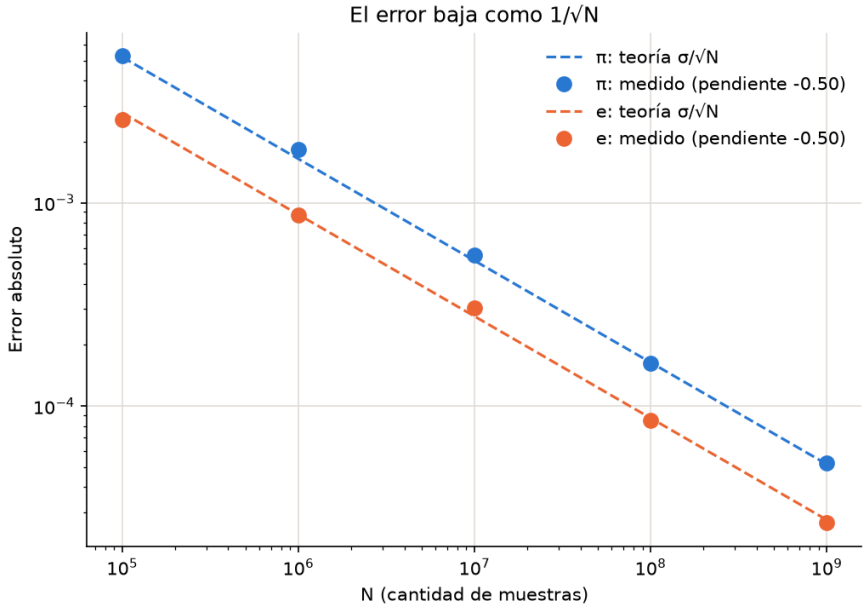


Figura 4. Error RMS en función de N en escala log-log (puntos), frente a la recta teórica $\sigma/\sqrt{N}$ (línea punteada). Se indica la pendiente ajustada para cada constante.

La aproximación normal también se verifica: el 95,2 % de las ejecuciones de π y el 95,7 % de las de e presentan un error menor que $2\sigma/\sqrt{N}$, frente al 95,4 % esperado. Con $N = 10^9$, el error típico es de aproximadamente $5 \cdot 10^{-5}$ para π y $3 \cdot 10^{-5}$ para e, es decir, cuatro decimales correctos. Ganar un decimal más requeriría $N = 10^{11}$, con un tiempo serial estimado de unos 20 minutos para π. Esta es la principal limitación del método y la razón por la cual la paralelización resulta necesaria.

Por último, la Figura 5 muestra el error normalizado $E_{abs}/(\sigma/\sqrt{N})$ para la versión serial y para cada cantidad de procesos. Todos los valores se encuentran entre 0,81 y 1,14, sin tendencia con P. Como cada valor resume 50 ejecuciones, su incertidumbre relativa es de aproximadamente 10 %, por lo que estas diferencias son compatibles con la fluctuación estadística esperada. Por lo tanto, paralelizar no degrada la precisión: el error depende de N y no de P, como se anticipó en la sección 4.3. Este resultado tampoco muestra indicios de correlación entre las secuencias aleatorias de los distintos procesos, aunque no alcanza para demostrar su independencia (sección 4.4).

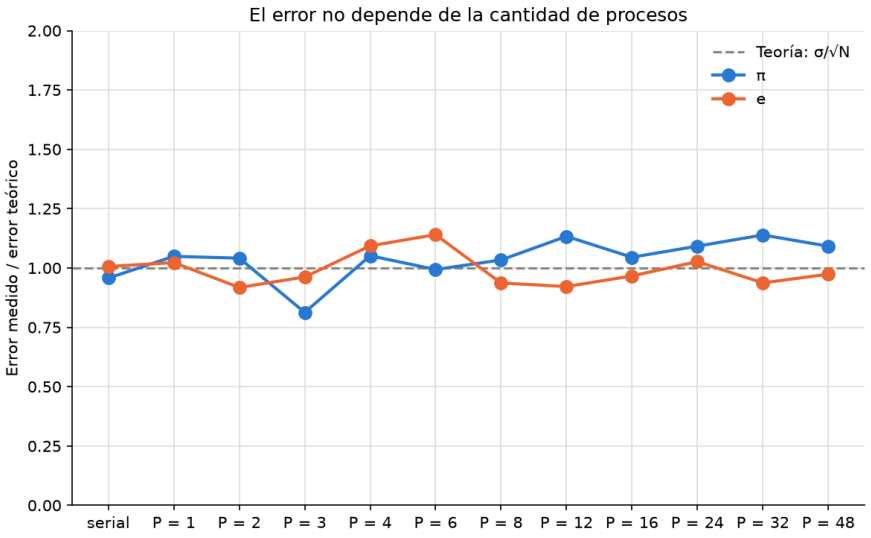


Figura 5. Error RMS normalizado por $\sigma/\sqrt{N}$ para la versión serial y para cada cantidad de procesos, combinando todos los valores de N. El valor 1 (línea punteada) corresponde a la predicción teórica.

6.2. Tiempos de ejecución

La Figura 6 muestra el tiempo mediano en función de N en el clúster, para la versión serial y para cada cantidad de procesos. En escala logarítmica, las curvas son rectas de pendiente 1: el tiempo crece linealmente con N, como corresponde a un algoritmo cuyo costo es proporcional a la cantidad de simulaciones, y cada duplicación de P desplaza la recta hacia abajo en un factor cercano a 2. Solo para N pequeños y muchos procesos las curvas se aplanan, porque los costos fijos de sincronización y comunicación se vuelven comparables con el cálculo.

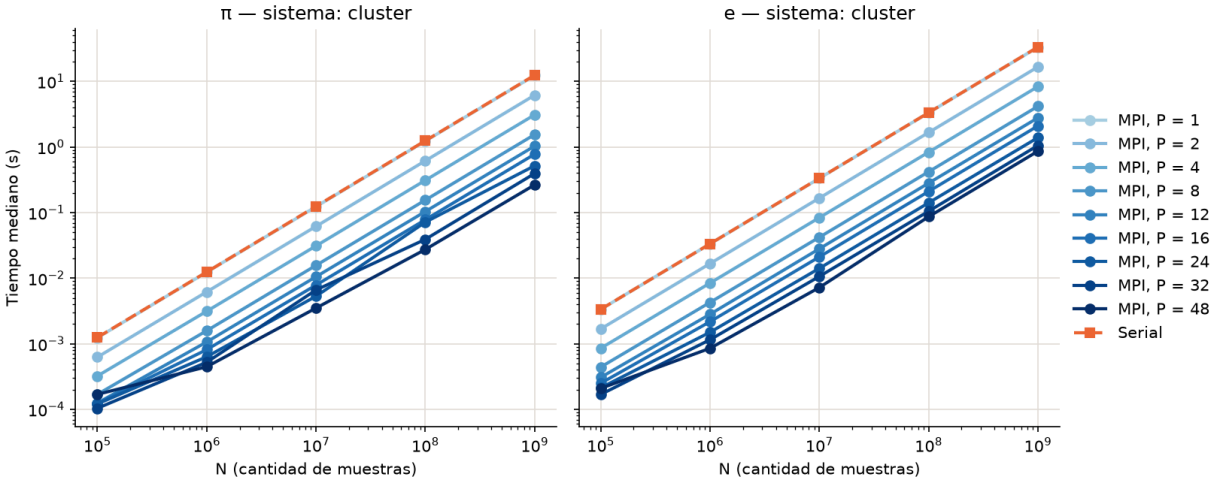


Figura 6. Tiempo mediano en función de N en el clúster, para la versión serial y para cada cantidad de procesos (escala log-log).

El costo por simulación es prácticamente igual en ambos sistemas: con $N = 10^9$, la versión serial tarda 12,4 s (clúster) y 12,1 s (PC) para π, es decir, unos 12 ns por punto, y 33,4 s y 34,1 s para e, unos 34 ns por simulación. Cada núcleo de ambos sistemas rinde, entonces, lo mismo para este

problema, y las diferencias entre sistemas que se analizan a continuación se deben a cómo escalan con P, no a la velocidad de cada núcleo.

Expresado por número aleatorio generado, π cuesta unos 6 ns (dos números por punto) y e unos 12 ns (2,72 números por simulación): cada número aleatorio cuesta el doble en e. Esta diferencia es consistente con la estructura de su bucle: la cantidad de iteraciones del bucle interno es aleatoria, lo que impide al procesador anticipar cuándo termina, y cada suma depende de la anterior. Este punto se retoma en las mejoras propuestas (sección 8).

Por último, en el clúster la versión MPI con un solo proceso tarda lo mismo que la versión serial (diferencias menores al 1,2 % para $N \geq 10^6$): la biblioteca MPI no agrega un costo apreciable al cálculo.

6.3. Speedup y eficiencia en el clúster

La Figura 7 muestra el *speedup* en el clúster para cada valor de N, y la Tabla 5 resume los valores obtenidos con $N = 10^9$ en ambos sistemas.

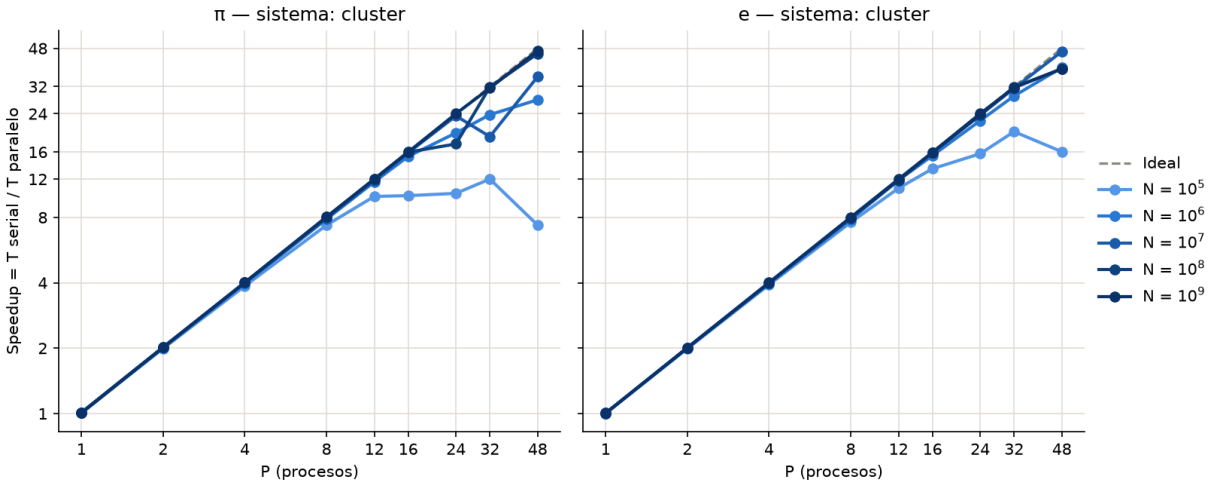


Figura 7. *Speedup* en el clúster en función de P para cada N (escala log-log). La línea punteada indica el *speedup* ideal.

Tabla 5. *Speedup* (S) y eficiencia (E) con $N = 10^9$ en ambos sistemas (mediana de 10 ejecuciones).

P	Clúster — π		Clúster — e		PC — π		PC — e	
	S	E	S	E	S	E	S	E
1	1,01	1,01	1,00	1,00	1,00	1,00	1,00	1,00
2	2,02	1,01	2,01	1,00	1,47	0,74	1,85	0,93
3	—	—	—	—	1,89	0,63	2,57	0,86
4	4,02	1,00	4,01	1,00	2,09	0,52	3,28	0,82
6	—	—	—	—	2,46	0,41	4,35	0,72
8	8,03	1,00	8,01	1,00	2,60	0,33	4,17	0,52
12	12,02	1,00	11,99	1,00	—	—	—	—
16	16,00	1,00	15,94	1,00	—	—	—	—
24	24,01	1,00	24,03	1,00	—	—	—	—
32	31,50	0,98	31,64	0,99	—	—	—	—
48	46,92	0,98	38,68	0,81	—	—	—	—

***N* grande.** Para $N \geq 10^8$, el *speedup* es prácticamente ideal hasta 32 procesos en ambas constantes, con eficiencias de 0,98 o superiores, y con 48 procesos π alcanza un *speedup* de 46,9 (eficiencia 0,98). Este comportamiento confirma lo esperado del diseño: los procesos no se comunican durante el cálculo y la única reducción final tiene un costo despreciable frente a segundos de cómputo.

e con 48 procesos. La mediana da un *speedup* de 38,7 (eficiencia 0,81). Sin embargo, los tiempos de las 10 ejecuciones no se distribuyen alrededor de un único valor, sino en dos grupos: 4 ejecuciones tardaron unos 0,70 s, lo que corresponde a un *speedup* de 47,8, prácticamente ideal, y 6 tardaron entre 0,86 y 0,90 s; la mediana queda en el grupo más lento. Con 48 procesos se ocupa la totalidad de la cuota de 48 CPU del contenedor, de modo que cualquier actividad adicional (el propio `mpirun`, los procesos auxiliares de Open MPI u otros procesos del contenedor) compite con el cálculo. Como el tiempo total lo determina el proceso más lento, ya que `MPI_Reduce` espera a todos, basta con que uno se demore para que se demore la ejecución completa. π con 48 procesos presenta el mismo fenómeno (3 de 10 ejecuciones lentas), pero allí la mayoría de las ejecuciones fue rápida y la mediana no se vio afectada. La diferencia entre π y e con $P = 48$ no es, entonces, una propiedad de los algoritmos, sino de cuántas ejecuciones resultaron afectadas en cada caso.

Interferencias con menos procesos. El mismo comportamiento bimodal aparece con 24 y 32 procesos, aunque en esos casos sobran CPU en la cuota. Por ejemplo, para π con $N = 10^9$ y $P = 24$, 6 ejecuciones tardaron 0,52 s y 4 tardaron entre 0,88 y 0,92 s. Una explicación posible es la interferencia de otros trabajos que comparten el nodo, ya que el contenedor ve los 128 núcleos de la máquina y los procesos no están fijados a núcleos (`--bind-to none`). Esto explica los dos puntos aislados de la Figura 7: π con $N = 10^8$ y $P = 24$ (eficiencia 0,73) y π con $N = 10^7$ y $P = 32$ (eficiencia 0,59), en los que la mitad o más de las ejecuciones fueron lentas. Como hay 10 repeticiones, la mediana es el promedio de los valores quinto y sexto, y puede quedar entre ambos grupos (por

ejemplo, 0,071 s entre 0,052 s y 0,091 s). Estos puntos no indican, por lo tanto, una tendencia. Cuando no hay interferencia, la variación entre repeticiones es del orden de 0,3 % o menor.

***N* pequeño.** Con $N = 10^5$, el *speedup* de π se estanca alrededor de 10 a 12 y cae a 7,4 con 48 procesos. Cada proceso realiza entonces unas 2.000 simulaciones, unas decenas de microsegundos de cálculo, del mismo orden que el costo de sincronizar y reducir los resultados. Es la manifestación práctica de la Ley de Amdahl (sección 3.5): la parte no paralelizable, que además crece con *P*, pesa más cuando el trabajo paralelo es pequeño. La estimación de e escala mejor con el mismo *N* (*speedup* de 19,9 con 32 procesos) porque cada simulación cuesta unas 2,7 veces más y, por lo tanto, cada proceso tiene más cálculo por cada comunicación. Para obtener eficiencias superiores a 0,9 con 48 procesos fueron necesarios $N = 10^8$ en π y $N = 10^7$ en e, es decir, del orden de $2 \cdot 10^6$ y $2 \cdot 10^5$ simulaciones por proceso, respectivamente.

6.4. Speedup y eficiencia en la PC

La Figura 8 muestra el *speedup* en la PC para $N \geq 10^7$. Los resultados son muy distintos de los del clúster: con $N = 10^9$, el *speedup* máximo es 2,60 para π (8 procesos) y 4,35 para e (6 procesos), y aun con 2 a 4 procesos, sin superar los núcleos físicos disponibles, la eficiencia es menor que 1, en especial en π (0,74 con solo 2 procesos; Tabla 5). Como el programa es el mismo que en el clúster escala de forma casi ideal, la causa debe buscarse en la plataforma y no en la paralelización. Se identifican los siguientes factores:

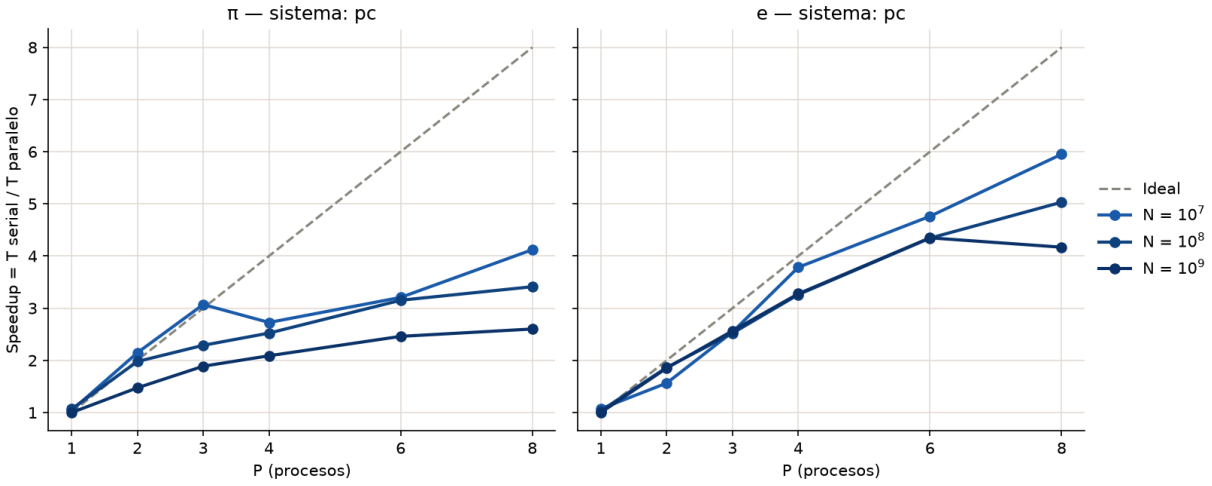


Figura 8. *Speedup* en la PC en función de *P* para $N \geq 10^7$. La línea punteada indica el *speedup* ideal.

- **Frecuencia variable del procesador.** El i5-10210U es un procesador de notebook de 15 W. Con un solo núcleo activo puede trabajar cerca de su frecuencia máxima (4,2 GHz), pero con varios núcleos activos el límite de potencia y temperatura obliga a reducirla. El tiempo serial de referencia se mide, entonces, a la frecuencia más alta y el paralelo a una menor, lo que reduce el *speedup* aunque la paralelización sea correcta.
- **Hyper-Threading.** A partir de 5 procesos, dos procesos comparten las unidades de ejecución de un mismo núcleo físico, y la ganancia depende de cuánto deja ocioso cada uno. La estimación de e todavía mejora de 4 a 6 procesos (de 3,28 a 4,35), mientras que π mejora poco (de 2,09 a 2,46).

Una explicación posible es que el bucle de e tiene más tiempos muertos (sección 6.2), que el otro hilo del núcleo puede aprovechar. Con 8 procesos, e empeora (4,17), porque se ocupan todos los núcleos lógicos y los procesos compiten con los del propio sistema.

- **Virtualización.** Las mediciones se realizaron en Linux sobre WSL 2, una máquina virtual de Windows. El sistema operativo anfitrión decide cómo se asignan los núcleos virtuales a los físicos y mantiene sus propios procesos en ejecución, lo que agrega variabilidad: la variación entre repeticiones llega al 12 %, frente a 0,3 % o menos en el clúster sin interferencia.

Con las mediciones disponibles no es posible cuantificar cuánto aporta cada factor, ni explicar con certeza por qué π escala peor que e con 2 a 4 procesos. Para hacerlo sería necesario registrar la frecuencia del procesador durante las ejecuciones o repetirlas con el Turbo Boost desactivado.

***N* pequeño.** Para $N \leq 10^6$ se obtienen eficiencias mayores que 1 (hasta 3,1 para e con $N = 10^5$), que no corresponden a un *speedup* superlineal real. Se trata de tiempos de pocos milisegundos, en los que la versión serial resulta notablemente más lenta que la versión MPI con un solo proceso (12,5 ms frente a 9,3 ms para e con $N = 10^5$), aunque ambas ejecutan el mismo cálculo. Como la versión serial es la primera de cada repetición, una posible causa es que el procesador todavía no haya alcanzado su frecuencia de trabajo al comenzar a medirla. Por lo tanto, en la PC solo se consideran interpretables los resultados con $N \geq 10^7$, que son los que muestra la Figura 8; la versión completa se incluye en el Anexo B (Figura B.2).

6.5. Comparación entre sistemas

Las Figuras 9 y 10 comparan el *speedup* y la eficiencia de ambos sistemas con $N = 10^9$. Aunque cada núcleo rinde lo mismo en los dos sistemas (sección 6.2), el clúster mantiene una eficiencia de 0,98 hasta 48 procesos en π, mientras que en la PC la eficiencia de π cae a 0,74 con 2 procesos y a 0,33 con 8, y la de e a 0,52 con 8.

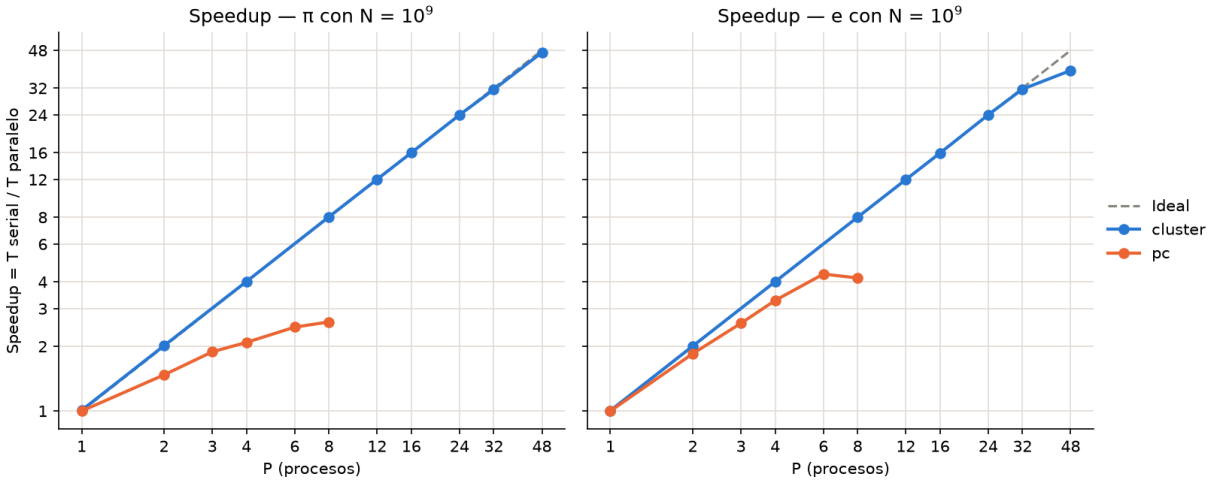


Figura 9. Comparación del *speedup* de ambos sistemas con $N = 10^9$ (escala log-log). La línea punteada indica el *speedup* ideal.

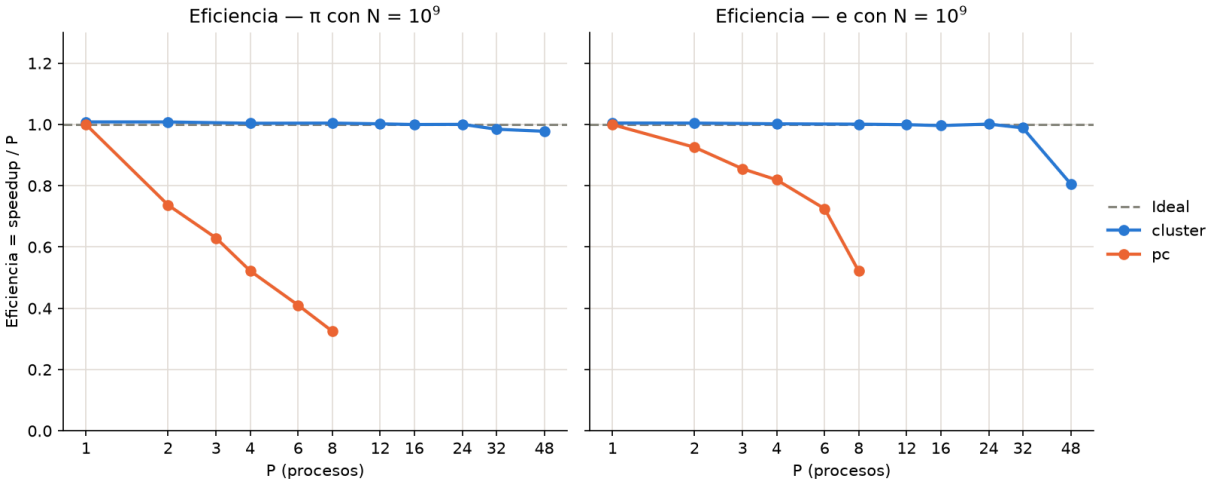


Figura 10. Comparación de la eficiencia de ambos sistemas con $N = 10^9$. La línea punteada indica la eficiencia ideal.

En tiempo absoluto, estimar π con $N = 10^9$ lleva 12,1 s en serie, 4,7 s en la PC con 8 procesos y 0,26 s en el clúster con 48 procesos, 47 veces menos que la versión serial. Para e, los tiempos son 34,1 s, 7,8 s (PC, 6 procesos) y 0,86 s (clúster, 48 procesos; 0,70 s en las ejecuciones sin interferencia).

La conclusión es que el *speedup* depende tanto de la plataforma como del algoritmo. Un problema trivialmente paralelo escala de forma casi ideal cuando cada proceso dispone de un núcleo físico dedicado y de una frecuencia estable; en una notebook, la frecuencia variable, los núcleos lógicos compartidos y la virtualización limitan el beneficio mucho antes de agotar el paralelismo del problema.

6.6. Limitaciones del estudio

- El tiempo medido excluye el lanzamiento de los procesos y la inicialización de MPI (sección 4.6). Para N pequeños, el tiempo total de una ejecución completa estaría dominado por esos costos.
- Las mediciones en el clúster se realizaron en un único nodo, por lo que no incluyen comunicación a través de la red entre nodos.
- Con 10 repeticiones, la mediana deja de ser robusta cuando la mitad o más de las ejecuciones resultan afectadas por interferencias (sección 6.3).
- En la PC no se registró la frecuencia del procesador, por lo que las causas de su menor escalabilidad se plantean como hipótesis.
- El esquema de semillas `semilla + rank` no garantiza formalmente la independencia entre las secuencias de los distintos procesos (sección 4.4).

7. Conclusiones

Se implementó la estimación de π y e mediante el Método de Monte Carlo en C, con una versión serial y una versión paralela con MPI, y se evaluó su precisión y su rendimiento en una computadora personal y en un nodo de clúster.

En cuanto a la precisión, los resultados experimentales coinciden con la teoría: el error disminuye como $1/\sqrt{N}$ (pendientes de −0,505 para π y −0,497 para e), su magnitud coincide con la predicción $\sigma/\sqrt{N}$ y cerca del 95 % de las ejecuciones queda dentro de dos errores típicos. Con $N = 10^9$ se obtienen cuatro decimales correctos de cada constante. La convergencia lenta es la principal limitación del método: cada decimal adicional requiere multiplicar por 100 la cantidad de simulaciones, y esta característica es la que justifica la paralelización.

La independencia entre las simulaciones permitió una paralelización con reparto estático y una única reducción al final, sin comunicación durante el cálculo. Se verificó, además, que repartir el trabajo no modifica la precisión: el error depende de N y no de la cantidad de procesos.

En el clúster, el *speedup* fue prácticamente ideal para N grandes, con eficiencias de 0,98 o superiores hasta 32 procesos en ambas constantes y hasta 48 procesos en π. Las caídas observadas en algunas configuraciones se explican por interferencias del entorno compartido, que afectan solo a una parte de las ejecuciones, y no por el algoritmo. Para N pequeños, el costo de sincronización y comunicación limita el *speedup*, de acuerdo con la Ley de Amdahl, y mantener una alta eficiencia requiere del orden de 10^5 a 10^6 simulaciones por proceso.

En la PC, el *speedup* fue mucho menor (2,6 para π y 4,4 para e), a pesar de que cada núcleo rinde lo mismo que en el clúster. La frecuencia variable de un procesador de bajo consumo, el uso de núcleos lógicos y la virtualización limitan el beneficio de la paralelización: el rendimiento paralelo depende tanto del algoritmo como de la plataforma en la que se ejecuta.

El uso de la mediana de múltiples ejecuciones resultó adecuado para obtener resultados estables, aunque en un entorno compartido conviene complementarla con más repeticiones o con el tiempo mínimo. Como trabajo futuro, el mayor margen de mejora se encuentra en reducir el costo de cada simulación y en garantizar la independencia de los números aleatorios entre procesos (sección 8).

8. Mejoras futuras

Los resultados muestran que la paralelización ya escala de forma casi ideal cuando el entorno lo permite. El margen de mejora no está, por lo tanto, en la distribución del trabajo, sino en tres aspectos: el costo de cada simulación, la validez estadística de los números aleatorios en paralelo y la información que el programa reporta sobre la incertidumbre de cada estimación. Las mejoras 8.1 a 8.4 se evaluaron en un banco de pruebas aparte (versión serial, en una notebook con Windows y gcc 8.1, compilada con `-O2`), pero no llegaron a integrarse al código del proyecto. Los tiempos que se informan son, por lo tanto, preliminares, y sirven para comparar las variantes entre sí, no con las mediciones de la sección 6 (Tabla 6).

8.1. Generador xoshiro256** con salto por proceso

Se propone reemplazar `drand48()` por el generador xoshiro256** [8]. Se trata de una función pequeña que el compilador puede insertar dentro del bucle (`static inline`), lo que evita una llamada a biblioteca por cada número y permite mantener el estado del generador en registros. Además, dispone de una función de salto (`jump`) que avanza la secuencia 2^{128} posiciones de una sola vez: si el proceso de rango r aplica el salto r veces sobre un estado inicial común, cada proceso recorre un tramo distinto de una secuencia de período $2^{256} - 1$, y la ausencia de solapamiento queda garantizada por construcción [9]. Esto resuelve la limitación descripta en la sección 4.4 y, en el banco de pruebas, redujo el costo de 5,4 a 3,3 ns por punto en π y de 18,6 a 14,6 ns por simulación en e.

8.2. Un número aleatorio por punto en π

Un único número de 64 bits alcanza para obtener las dos coordenadas del punto, 31 bits cada una, lo que reduce a la mitad las llamadas al generador (de 3,3 a 2,5 ns por punto). El sesgo que introduce discretizar cada coordenada en 2^{31} valores es del orden de 10^{-9}, varios órdenes de magnitud menor que el error estadístico de cualquiera de las ejecuciones realizadas.

8.3. Aritmética entera

En π, la condición $x^2 + y^2 \le 1$ puede evaluarse con enteros ($X^2 + Y^2 \le 2^{62}$) sin convertir a `double`; la ganancia allí es pequeña (de 2,5 a 2,3 ns, dentro del ruido de la medición). En e, la suma acumulada puede representarse como un entero de 53 bits, con el umbral 1 equivalente a 2^{53}. La suma entera es exacta y más rápida que la de punto flotante, lo que resulta importante porque cada suma depende de la anterior.

8.4. Bucle sin saltos impredecibles en e

En la sección 6.2 se observó que cada número aleatorio cuesta el doble en e que en π. La causa principal es el bucle interno `while (suma <= 1.0)`, cuya cantidad de iteraciones es aleatoria: el procesador no puede anticipar cuándo termina, y casi cada simulación finaliza con una predicción de salto fallida. Se propone reemplazar los dos bucles anidados por uno solo, que reinicia la suma mediante una operación aritmética (una máscara) en lugar de una condición. El estimador no

cambia, ya que se consumen los mismos números en el mismo orden. Combinada con la mejora 8.3, el costo bajó de 14,6 a 6,2 ns por simulación, por lo que es la mejora de mayor impacto.

8.5. Error estándar en la salida

Actualmente cada ejecución informa su error absoluto, que solo puede calcularse porque se conoce el valor real. Se propone informar también el error estándar estimado a partir de las propias muestras:

$$\mathrm{SE}_\pi = 4\sqrt{\frac{\hat{p}\,(1-\hat{p})}{N}}, \quad \hat{p} = \frac{\text{aciertos}}{N} \qquad \mathrm{SE}_e = \sqrt{\frac{s^2}{N}}, \quad s^2 = \frac{1}{N}\sum K_i^2 - \hat{e}_N^{\,2} \tag{14}$$

En e, esto requiere acumular también la suma de K^2 y reducir un segundo valor. De este modo, cada ejecución informaría su propio intervalo de confianza, como se haría en un problema real donde el valor exacto es desconocido. La verificación de la sección 6.1 (95 % de las ejecuciones dentro de dos errores típicos) se realizó a posteriori con la varianza teórica; con esta mejora pasaría a formar parte de la salida del programa.

Tabla 6. Efecto acumulado de las mejoras propuestas en el banco de pruebas (versión serial, ns por iteración, `-O2`; menos es mejor).

Paso	π	e
Código actual (`drand48`)	5,4	18,6
+ xoshiro256** (8.1)	3,3	14,6
+ un número por punto (8.2)	2,5	—
+ aritmética entera (8.3)	2,3	9,8
+ bucle plano con enteros (8.3 y 8.4)	—	6,2
Mejora total	**≈ 2,3×**	**≈ 3×**

Efecto esperado sobre el *speedup*. Estas mejoras aceleran por igual la versión serial y la paralela, por lo que no deberían modificar el *speedup* relativo, sino reducir el tiempo absoluto. Para una comparación honesta, el tiempo de referencia T_s debería medirse con la versión serial mejorada. Además, al reducirse el tiempo de cálculo, los costos fijos de comunicación pesarán más para N pequeños, por lo que serán necesarios valores de N mayores para mantener la eficiencia observada.

8.6. Mejoras en el procedimiento experimental

Los resultados de la sección 6 sugieren, además: aumentar la cantidad de repeticiones o informar también el tiempo mínimo y el rango intercuartílico, para detectar distribuciones bimodales como las observadas en el clúster; fijar cada proceso a un núcleo y dejar al menos una CPU libre de la cuota (por ejemplo, medir con 47 procesos) para reducir interferencias; registrar la frecuencia del procesador en la PC; y medir también el tiempo de la ejecución completa, incluyendo el lanzamiento de los procesos.

8.7. Otras alternativas consideradas

Se evaluaron otras mejoras habituales que no se priorizaron. El modelo híbrido MPI + OpenMP no aportaría ganancia dentro de un único nodo, donde los procesos MPI ya ocupan todos los núcleos; solo tendría sentido al escalar a varios nodos. Las técnicas de reducción de varianza (variables antitéticas, muestreo estratificado o cuasi-Monte Carlo) pueden mejorar la precisión para un mismo N, pero modifican el método de estimación, por lo que corresponderían a una variante a comparar y no a un reemplazo. La optimización del uso de la memoria caché no aplica, porque el programa no trabaja con estructuras de datos en memoria: su costo es de cálculo, no de acceso a memoria. Por último, la vectorización SIMD y la aceleración mediante GPU son posibles, pero requieren reescribir el núcleo de cálculo y quedan como líneas de trabajo de mayor alcance.

9. Bibliografía

[1] N. Metropolis y S. Ulam, “The Monte Carlo method,” *Journal of the American Statistical Association*, vol. 44, n.º 247, pp. 335–341, 1949.

[2] C. P. Robert y G. Casella, *Monte Carlo Statistical Methods*, 2.ª ed. Nueva York: Springer, 2004.

[3] K. G. Russell, “Estimating the value of e by simulation,” *The American Statistician*, vol. 45, n.º 1, pp. 66–68, 1991.

[4] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” en *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[5] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011.

[6] The Open Group, “drand48, erand48, jrand48, lcong48, lrand48, mrand48, nrand48, seed48, srand48 — generate uniformly distributed pseudo-random numbers,” *The Open Group Base Specifications*, Issue 7 (IEEE Std 1003.1-2017).

[7] Linux man-pages project, “drand48(3) — Linux manual page.” [En línea]. Disponible: https://man7.org/linux/man-pages/man3/drand48.3.html

[8] D. Blackman y S. Vigna, “Scrambled linear pseudorandom number generators,” *ACM Transactions on Mathematical Software*, vol. 47, n.º 4, art. 36, 2021.

[9] P. L’Ecuyer, D. Munger, B. Oreshkin y R. Simard, “Random numbers for parallel computers: Requirements and methods, with emphasis on GPUs,” *Mathematics and Computers in Simulation*, vol. 135, pp. 3–17, 2017.

[10] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, versión 4.0, 2021.

[11] Open MPI Project, “Open MPI Documentation.” [En línea]. Disponible: https://www.open-mpi.org/doc/

Anexo A. Tabla completa de resultados

Mediana del tiempo de 10 ejecuciones (en segundos), *speedup* S y eficiencia E para cada configuración. T_s es la mediana del tiempo de la versión serial para el mismo *N*. Los datos completos de cada ejecución se encuentran en el repositorio (`resultados/`).

Tabla A.1. Clúster — π.

N	*P*	T_P (s)	T_s (s)	S	E
10^5	1	0,0012	0,0012	1,01	1,01
10^5	2	0,0006	0,0012	1,99	1,00
10^5	4	0,0003	0,0012	3,87	0,97
10^5	8	0,0002	0,0012	7,37	0,92
10^5	12	0,0001	0,0012	9,99	0,83
10^5	16	0,0001	0,0012	10,07	0,63
10^5	24	0,0001	0,0012	10,32	0,43
10^5	32	0,0001	0,0012	12,01	0,38
10^5	48	0,0002	0,0012	7,37	0,15
10^6	1	0,0124	0,0124	1,00	1,00
10^6	2	0,0062	0,0124	2,01	1,00
10^6	4	0,0031	0,0124	3,97	0,99
10^6	8	0,0016	0,0124	7,85	0,98
10^6	12	0,0011	0,0124	11,65	0,97
10^6	16	0,0008	0,0124	15,27	0,95
10^6	24	0,0006	0,0124	19,59	0,82
10^6	32	0,0005	0,0124	23,76	0,74
10^6	48	0,0004	0,0124	27,84	0,58
10^7	1	0,1231	0,1245	1,01	1,01
10^7	2	0,0617	0,1245	2,02	1,01
10^7	4	0,0311	0,1245	4,01	1,00
10^7	8	0,0157	0,1245	7,95	0,99
10^7	12	0,0105	0,1245	11,83	0,99
10^7	16	0,0078	0,1245	15,87	0,99
10^7	24	0,0053	0,1245	23,46	0,98
10^7	32	0,0066	0,1245	18,84	0,59
10^7	48	0,0035	0,1245	35,56	0,74
10^8	1	1,2320	1,2447	1,01	1,01
10^8	2	0,6158	1,2447	2,02	1,01
10^8	4	0,3106	1,2447	4,01	1,00
10^8	8	0,1551	1,2447	8,03	1,00
10^8	12	0,1038	1,2447	11,99	1,00
10^8	16	0,0779	1,2447	15,97	1,00
10^8	24	0,0714	1,2447	17,43	0,73
10^8	32	0,0391	1,2447	31,85	1,00
10^8	48	0,0274	1,2447	45,42	0,95
10^9	1	12,3243	12,4243	1,01	1,01
10^9	2	6,1626	12,4243	2,02	1,01
10^9	4	3,0944	12,4243	4,02	1,00
10^9	8	1,5463	12,4243	8,03	1,00
10^9	12	1,0334	12,4243	12,02	1,00
10^9	16	0,7768	12,4243	15,99	1,00
10^9	24	0,5175	12,4243	24,01	1,00
10^9	32	0,3944	12,4243	31,50	0,98
10^9	48	0,2648	12,4243	46,92	0,98

Tabla A.2. Clúster — e.

N	*P*	T_P (s)	T_s (s)	S	E
10^5	1	0,0034	0,0034	1,00	1,00
10^5	2	0,0017	0,0034	1,99	1,00
10^5	4	0,0009	0,0034	3,92	0,98
10^5	8	0,0004	0,0034	7,59	0,95
10^5	12	0,0003	0,0034	10,89	0,91
10^5	16	0,0003	0,0034	13,39	0,84
10^5	24	0,0002	0,0034	15,71	0,65
10^5	32	0,0002	0,0034	19,85	0,62
10^5	48	0,0002	0,0034	16,05	0,33
10^6	1	0,0333	0,0335	1,00	1,00
10^6	2	0,0167	0,0335	2,00	1,00
10^6	4	0,0084	0,0335	3,97	0,99
10^6	8	0,0043	0,0335	7,82	0,98
10^6	12	0,0028	0,0335	11,85	0,99
10^6	16	0,0022	0,0335	15,39	0,96
10^6	24	0,0015	0,0335	22,30	0,93
10^6	32	0,0012	0,0335	28,98	0,91
10^6	48	0,0009	0,0335	39,15	0,82
10^7	1	0,3330	0,3345	1,00	1,00
10^7	2	0,1665	0,3345	2,01	1,00
10^7	4	0,0838	0,3345	3,99	1,00
10^7	8	0,0421	0,3345	7,95	0,99
10^7	12	0,0281	0,3345	11,90	0,99
10^7	16	0,0211	0,3345	15,84	0,99
10^7	24	0,0141	0,3345	23,65	0,99
10^7	32	0,0106	0,3345	31,45	0,98
10^7	48	0,0072	0,3345	46,38	0,97
10^8	1	3,3288	3,3400	1,00	1,00
10^8	2	1,6647	3,3400	2,01	1,00
10^8	4	0,8344	3,3400	4,00	1,00
10^8	8	0,4198	3,3400	7,96	0,99
10^8	12	0,2815	3,3400	11,87	0,99
10^8	16	0,2102	3,3400	15,89	0,99
10^8	24	0,1411	3,3400	23,67	0,99
10^8	32	0,1050	3,3400	31,82	0,99
10^8	48	0,0866	3,3400	38,56	0,80
10^9	1	33,2767	33,4297	1,00	1,00
10^9	2	16,6423	33,4297	2,01	1,00
10^9	4	8,3407	33,4297	4,01	1,00
10^9	8	4,1750	33,4297	8,01	1,00
10^9	12	2,7883	33,4297	11,99	1,00
10^9	16	2,0975	33,4297	15,94	1,00
10^9	24	1,3913	33,4297	24,03	1,00
10^9	32	1,0565	33,4297	31,64	0,99
10^9	48	0,8643	33,4297	38,68	0,81

Tabla A.3. PC — π.

N	P	T_P (s)	T_s (s)	S	E
10^5	1	0,0014	0,0017	1,25	1,25
10^5	2	0,0006	0,0017	2,67	1,34
10^5	3	0,0004	0,0017	4,07	1,36
10^5	4	0,0003	0,0017	5,18	1,30
10^5	6	0,0004	0,0017	4,09	0,68
10^5	8	0,0003	0,0017	5,11	0,64
10^6	1	0,0139	0,0151	1,09	1,09
10^6	2	0,0061	0,0151	2,49	1,24
10^6	3	0,0042	0,0151	3,61	1,20
10^6	4	0,0031	0,0151	4,91	1,23
10^6	6	0,0040	0,0151	3,77	0,63
10^6	8	0,0032	0,0151	4,77	0,60
10^7	1	0,1229	0,1278	1,04	1,04
10^7	2	0,0595	0,1278	2,15	1,07
10^7	3	0,0416	0,1278	3,07	1,02
10^7	4	0,0469	0,1278	2,73	0,68
10^7	6	0,0398	0,1278	3,21	0,53
10^7	8	0,0310	0,1278	4,12	0,52
10^8	1	1,1926	1,2705	1,07	1,07
10^8	2	0,6424	1,2705	1,98	0,99
10^8	3	0,5552	1,2705	2,29	0,76
10^8	4	0,5036	1,2705	2,52	0,63
10^8	6	0,4030	1,2705	3,15	0,53
10^8	8	0,3724	1,2705	3,41	0,43
10^9	1	12,1450	12,1382	1,00	1,00
10^9	2	8,2368	12,1382	1,47	0,74
10^9	3	6,4304	12,1382	1,89	0,63
10^9	4	5,8120	12,1382	2,09	0,52
10^9	6	4,9339	12,1382	2,46	0,41
10^9	8	4,6664	12,1382	2,60	0,33

Tabla A.4. PC — e.

N	P	T_P (s)	T_s (s)	S	E
10^5	1	0,0093	0,0125	1,35	1,35
10^5	2	0,0051	0,0125	2,47	1,24
10^5	3	0,0021	0,0125	5,82	1,94
10^5	4	0,0010	0,0125	12,31	3,08
10^5	6	0,0010	0,0125	12,31	2,05
10^5	8	0,0009	0,0125	14,51	1,81
10^6	1	0,0717	0,1030	1,44	1,44
10^6	2	0,0380	0,1030	2,71	1,35
10^6	3	0,0230	0,1030	4,47	1,49
10^6	4	0,0172	0,1030	5,99	1,50
10^6	6	0,0118	0,1030	8,73	1,46
10^6	8	0,0078	0,1030	13,27	1,66
10^7	1	0,4387	0,4703	1,07	1,07
10^7	2	0,3008	0,4703	1,56	0,78
10^7	3	0,1849	0,4703	2,54	0,85
10^7	4	0,1243	0,4703	3,78	0,95
10^7	6	0,0988	0,4703	4,76	0,79
10^7	8	0,0790	0,4703	5,95	0,74
10^8	1	3,4054	3,5162	1,03	1,03
10^8	2	1,8874	3,5162	1,86	0,93
10^8	3	1,3948	3,5162	2,52	0,84
10^8	4	1,0783	3,5162	3,26	0,82
10^8	6	0,8097	3,5162	4,34	0,72
10^8	8	0,6988	3,5162	5,03	0,63
10^9	1	34,0833	34,0598	1,00	1,00
10^9	2	18,4035	34,0598	1,85	0,93
10^9	3	13,2776	34,0598	2,57	0,86
10^9	4	10,3941	34,0598	3,28	0,82
10^9	6	7,8322	34,0598	4,35	0,72
10^9	8	8,1691	34,0598	4,17	0,52

Anexo B. Gráficos complementarios

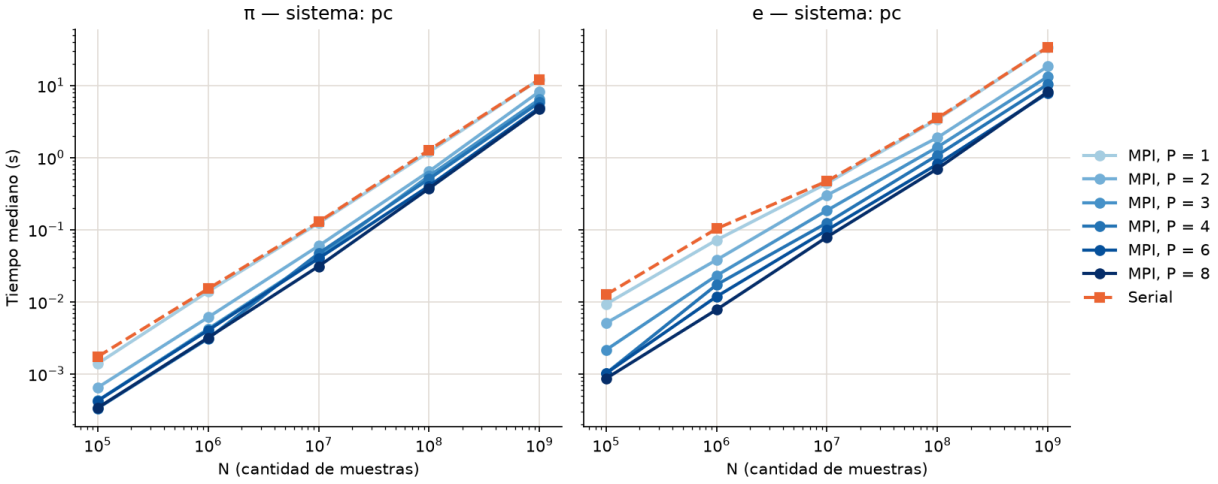


Figura B.1. Tiempo mediano en función de N en la PC, para la versión serial y para cada cantidad de procesos (escala log-log).

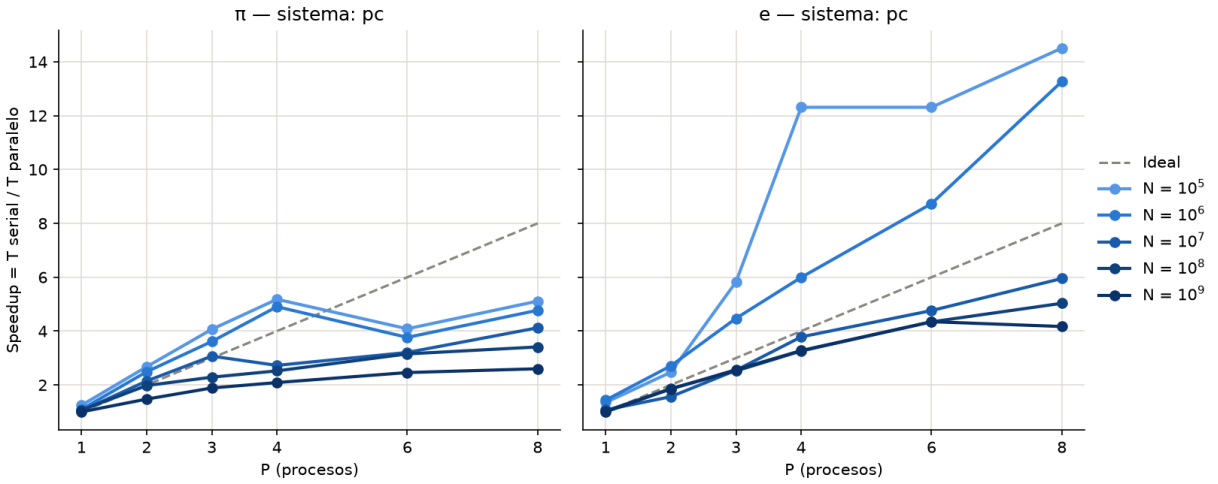


Figura B.2. *Speedup* en la PC en función de P para todos los valores de N. Los valores superiores al ideal para $N \leq 10^6$ son artefactos de medición (sección 6.4).

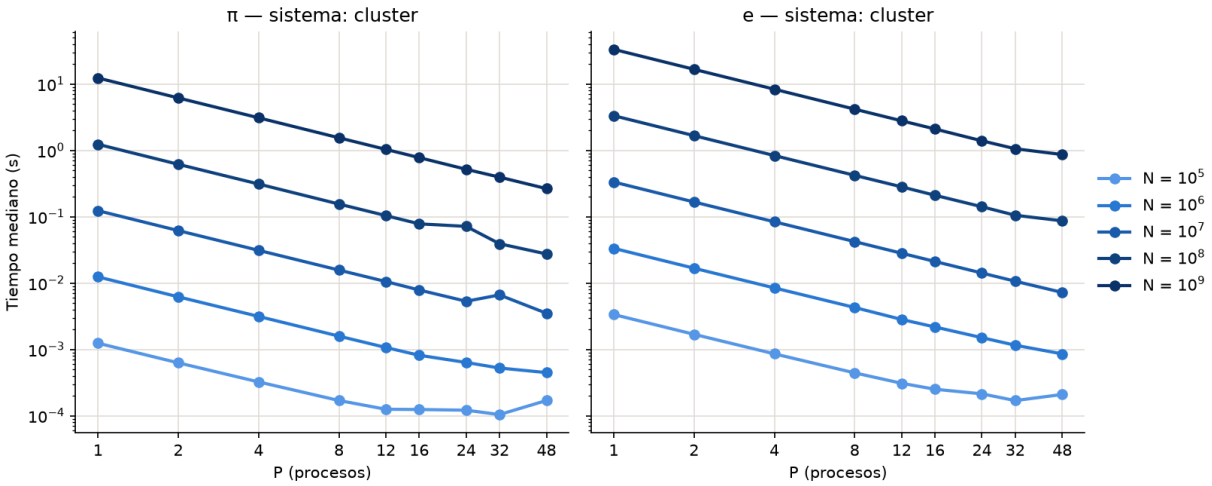


Figura B.3. Tiempo mediano en función de *P* en el clúster para cada *N* (escala logarítmica en el tiempo).

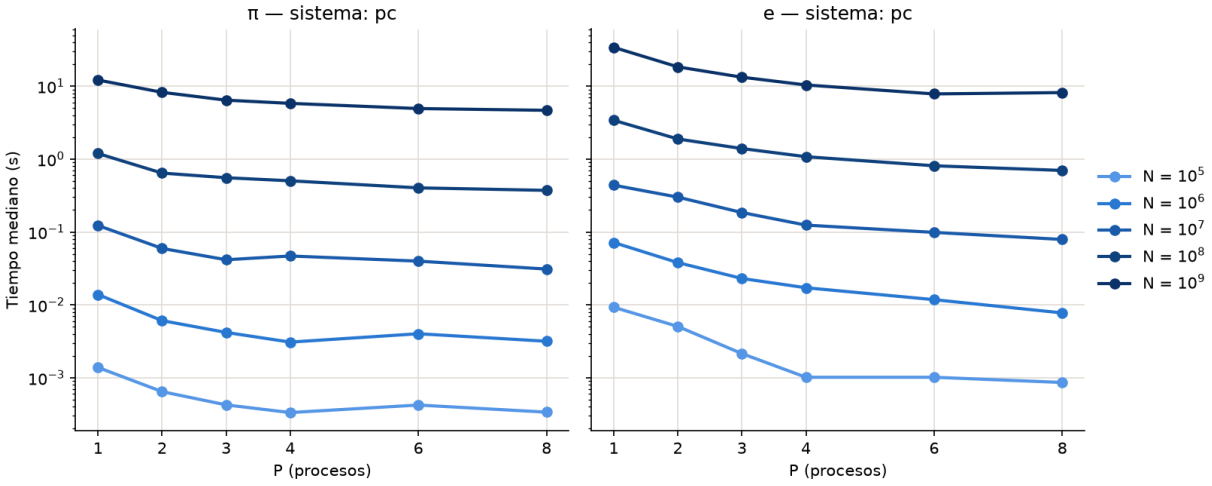


Figura B.4. Tiempo mediano en función de *P* en la PC para cada *N* (escala logarítmica en el tiempo).